Functional vs. Object-Oriented Programming

in the context of Lean 4

The short version

Lean 4 is a **purely functional** language. It has no classes, no objects with hidden mutable state, and no subclass polymorphism. Yet many OOP ideas have a functional counterpart in Lean:

OOP concept	Lean counterpart
Class with fields	`structure`
Methods	Functions in the structure's `namespace`, called with dot notation
Inheritance	`structure B extends A` (composition + automatic projection)
Interface / abstract class	Type class (`class` + `instance`)
Class hierarchy (closed)	`inductive` type + pattern matching
Mutating a field	Building a new value: `{ p with x := ... }`
Mutable state	Monads (`StateM`, `IO.Ref`) and `let mut` sugar

The key difference: in Lean every function is a *mathematical* function. Same input, same output, no side effects. That is exactly what lets Lean **prove theorems about programs**.

1 Data and behaviour

OOP bundles data and methods into one class; methods act on `this`.

```
class Point {
  int x, y;
  int normSq()              { return x*x + y*y; }
  void move(int dx, int dy) { x += dx; y += dy; }   // mutates this
}
```

Lean separates data (`structure`) from functions. Putting the functions in the `Point` namespace enables dot notation, so it *looks* like a method call, but `p.normSq` is just `Point.normSq p`.

```
structure Point where
  x : Int
  y : Int

namespace Point
def normSq (p : Point) : Int := p.x * p.x + p.y * p.y
def move (p : Point) (dx dy : Int) : Point :=
  { p with x := p.x + dx, y := p.y + dy }   -- returns a NEW point
end Point

def p : Point := { x := 3, y := 4 }
#eval p.normSq          -- 25
#eval (p.move 1 1).x    -- 4
#eval p.x               -- 3   (p is unchanged)
```

2 Inheritance vs. structure extension

OOP: `class ColoredPoint extends Point`. A `ColoredPoint` *is a* `Point` and can override its methods.

Lean: `extends` copies the parent's fields and generates a projection `toPoint`. Dot notation searches the parent namespace, so parent functions still work. There is *no overriding* and no dynamic dispatch: this is composition, not subtyping.

```
structure ColoredPoint extends Point where
  color : String

def cp : ColoredPoint := { x := 1, y := 2, color := "red" }
#eval cp.normSq        -- 5   (resolved as Point.normSq cp.toPoint)
```

3 Polymorphism

OOP uses one mechanism (virtual methods) for two different needs. Lean splits them.

Open set of types → type classes

Like an interface: anyone can add a new type later by writing an instance. Dispatch is resolved at *compile time* by instance search, not at run time via a vtable.

```
class HasArea (α : Type) where
  area : α → Nat

structure Square where side : Nat
structure Rect   where (w h : Nat)

instance : HasArea Square where area s := s.side * s.side
instance : HasArea Rect   where area r := r.w * r.h

def totalArea {α : Type} [HasArea α] (xs : List α) : Nat :=
  (xs.map HasArea.area).sum

#eval totalArea [Square.mk 2, Square.mk 3]   -- 13
```

Closed set of cases → inductive types

Like a sealed class hierarchy. All variants are known up front, and pattern matching must be *exhaustive*, so the compiler rejects a forgotten case.

```
inductive Shape where
  | square (side : Nat)
  | rect   (w h : Nat)

def Shape.area : Shape → Nat
  | .square s => s * s
  | .rect w h => w * h

#eval ([.square 2, .rect 2 5] : List Shape).map Shape.area   -- [4, 10]
```

Trade-off (the "expression problem"): OOP makes adding a new *type* easy and a new *operation* hard. Inductive types make adding a new *operation* easy (just write a function) and a new *case* hard (every match must be updated).

4 State and mutation

OOP objects carry mutable state. Lean values are immutable; state is *modelled explicitly*. The `do`-notation with `let mut` looks imperative, but it compiles to pure code.

```
def sumTo (n : Nat) : Nat := Id.run do
  let mut acc := 0
  for i in [0:n+1] do
    acc := acc + i
  return acc
#eval sumTo 10                    -- 55

-- A "counter object" as an explicit state monad
def tick : StateM Nat Unit := modify (· + 1)
def threeTicks : StateM Nat Unit := do tick; tick; tick
#eval threeTicks.run 0            -- ((), 3)
```

5 Why it matters: proofs

Since functions are pure, they can be reasoned about like equations. The same file that defines `Point.move` can prove facts about it, checked by Lean's kernel.

```
theorem move_zero (q : Point) : q.move 0 0 = q := by
  simp [Point.move]

theorem area_square (n : Nat) : (Shape.square n).area = n * n := rfl
```

In an OOP setting, `p.move(0,0)` could in principle touch any reachable state (logging, other objects, globals), so a statement like the one above would not even be well defined without extra assumptions.

6 Summary

	OOP	**Lean (functional)**
Core unit	Object (state + behaviour)	Value + pure function
State	Mutable, hidden in objects	Immutable; explicit via monads
Reuse	Inheritance	`extends`, type classes, composition
Dispatch	Dynamic (run time)	Static (instances) or pattern matching
Extensibility	Easy new types	Easy new operations (inductives)
Correctness	Tests	Tests *and* machine-checked proofs

All Lean snippets typecheck with Lean 4.35.0 (core only, no Mathlib).